Software Requirement

Definition:

- According to IEEE Standard, Software Requirement is a **condition needed by a user to solve a problem** or achieve an objective.
- Requirements should be **clear, correct and well-defined.**

Types of Requirements:

There are two types of requirements:

1. **Functional Requirements**
2. **Non-Functional Requirements**

A common way to remember it is that **Functional** is about the "What" and **Non-Functional** is about the "How."

1. Functional Requirements (The "What")

These define the specific features and behaviors of the software. If these aren't met, the system simply cannot perform the tasks it was built for.

- **Focus:** User tasks, data processing, and system behavior.
- **Examples:**
    - **Login:** The system must allow users to log in using a username and password.
    - **Payment:** The app must process credit card payments via Stripe.
    - **Search:** Users must be able to search for products by category or price range.

2. Non-Functional Requirements (The "How")

These define the quality attributes or constraints of the system. They describe how the system should perform its functions.

- **Focus:** Performance, security, usability, and reliability.
- **Examples:**
    - **Speed:** The login page must load within 2 seconds.
    - **Capacity:** The system should support 500 concurrent users without crashing.

- **Security:** All user passwords must be encrypted using AES-256.

Comparison:

Feature	Functional Requirement	Non-Functional Requirement
Question	What does the system do?	How does the system perform?
Requirement	Mandatory (Essential features)	Quality (User experience/Performance)
Testing	Functional testing (Unit, Integration)	Performance, Security, Usability testing
Failure	The system is incomplete.	The system is slow, insecure, or hard to use.

Requirement Engineering

The process to **gather the software requirements from customers, analyze and document them** is known as requirement engineering.

Requirement engineering provides the appropriate mechanism to understand:

- Customer desires / wishes
- Analyzing Customer need
- Assessing feasibility
- Negotiating a reasonable solution
- Specifying the solution clearly
- Validating the specifications and managing the requirements

as they are transformed into a working system

No.	Users Requirements	System Requirements
1	Written for **Customers**.	Written for **Implementation Team**.
2	In **Natural language**.	In **Technical Language**.
3	Describe the **services & features** provide by system.	Describe the **detail description** of services, features & complete operations of system.
4	May include **diagrams & tables**.	May include **system models**.

5	Understandable by **system users** who don't have technical knowledge.	Understandable by **implementation team** who have technical knowledge.
6	For Client Managers, System Users, Contractor Managers, System Architects.	For System Architects, Software Developers, Client Engineers, System Users.

Tools in Requirement Engineering

1. Observation report
2. Questionnaire (survey, poll)
3. Use cases
4. User stories
5. Requirement workshop
6. Mind mapping
7. Role playing
8. Prototyping

Available Software's:

1. Xebrio – Requirements Management Software
2. Jira for requirements engineering
3. Doc Sheets
4. RequirementsHub
5. IBM Engineering Requirements Management DOORS
6. Modern Requirements
7. SpiraTeam

Establishing Groundwork

1. Identifying the stakeholders:

Any person who benefits directly or indirectly from the **system** being developed is a stakeholder.

Examples: Business operations managers, product managers, marketing people, internal and external customers, end-users, consultants, product engineers, software engineers, developers, testers, and support/maintenance engineers are the usual stakeholders.

Each stakeholder sees the system differently, gains different benefits when the system is successfully developed.

They also **face different risks** if the development effort fails.

2. Recognizing Multiple Viewpoints:

Because there are so many different stakeholders, the system's requirements will be examined from **various perspectives**.

Each of these stakeholders will **contribute data** to the requirements engineering process.

As information is gathered from multiple viewpoints, **collecting requirements may be inconsistent or contradictory**.

You should **categorize all stakeholder information** so that decision-makers can select a consistent set of system requirements for the system.

Example-

The Conflict: The **End-User** wants "one-click" transfers (for speed), but the **Security Engineer** requires a 6-digit PIN and a face scan for every transaction (for safety).

3. Working toward Collaboration:

If there are five stakeholders involved in a software project, there may be **five different opinions on the set of requirements**.

Customers must work together with software development team to create a successful system.

A requirements engineer's job is **to identify areas of commonality requirement as well as areas of conflict requirements**.

At the end **project head & requirement engineer may decide on which requirements are accepted**.

4. Asking the first question: (For identifying stakeholders)

The first set of questions focuses on the **customers and stakeholders as well as the overall project goals and benefits**.

1. **Who is the person or organization** behind the request for this work?
2. **Who will make use** of the solution?
3. What is the **economic value** of a successful solution?
4. Is there a **different source** for the solution you require?

Asking the first questions: (Understanding Problems & Solution)

Following set of questions helps in gaining a **better understanding of the problem and allows the customer to express his or her thoughts** on a solution:

1. How would you characterize **"good" output** produced by a successful solution?
2. To **what problem(s)** will this solution be applied?
3. Can you **describe (or show me) the business environment** in which the solution will be used?
4. Will there be any **special performance issues or constraints** that will influence how the solution is approached?

Requirement Engineering Tasks

Requirement engineering constructs a **bridge for design and construction**.

Requirement Engineering consists of seven different tasks:

1. Inception
2. Elicitation
3. Elaboration
4. Negotiation
5. Specification
6. Validation
7. Management

1. Inception

Understanding **why** the software is needed and **who** will use it.

Example:
A school says, *"We want software to manage student records."*
→ You identify the purpose and users (teachers, students, admin).

2. Elicitation

Collecting requirements from users by asking questions.

Example:
You ask teachers:
"Do you need attendance, marks, or report cards?"

3. Elaboration

Explaining requirements in detail and organizing them clearly.

Example:
Instead of "attendance system," you define:
Daily attendance, monthly report, absent list.

4. Negotiation

Resolving conflicts when users want different things.

Example:
Admin wants many features, but budget is low → decide **which features are most important**.

5. Specification

Writing all requirements formally in a document.

Example:
A document that says:
"The system shall allow teachers to enter marks online."

6. Validation

Checking if requirements are correct and complete.

Example:
You show the document to teachers and ask:
"Is this what you really want?"

7. Management

Handling changes in requirements over time.

Example:
Later, the school asks:
"Add online fee payment."
→ You update and track the change.

Requirement Engineering Process

Following **four steps** involved in Requirement Engineering Process:

Feasibility Study

1. Requirement Elicitation and Analysis
2. Software Requirement Specification
3. Software Requirement Validation
4. Software Requirement Management

Feasibility Study

- To **create & analyze the reasons for developing the software** that is acceptable to users.
- To check **aim & goal of the customer & organization** behind implementation of software.

Types of Feasibility Study:

1. **Technical Feasibility:** Evaluates the **current technologies**, which are needed to achieve customer requirements **within the time and budget**.
2. **Operational Feasibility:** Required software **solve business problems** and meets customer requirements.
3. **Economic Feasibility:** It decides whether the necessary software can **generate financial profits** for an organization.

The output is to decide whether or not the project should be undertaken.

Step 1: Requirement Elicitation & Analysis

- Requirements elicitation is the **process of gathering actual requirements about the needs and expectations of stakeholders** for a software system.
- To understand **customers, business manuals, the existing software of same type, standards and other stakeholders** of the project.

Techniques of Requirements Elicitation:

1. **Interviews: One-on-one conversations** with stakeholders to gather information about their needs and expectations.
2. **Surveys:** These are **questionnaires** that are distributed to stakeholders to gather information.
3. **Focus Groups: Small groups of stakeholders** who are brought together to discuss their needs.
4. **Observation: Observing the stakeholders in their work environment** to gather information.
5. **Prototyping: Creating a working model of the software system**, which can be used to **gather feedback** from stakeholders and to validate requirements.

Step 2: Software Requirement Specification (SRS)

- Requirements received from **client are written in natural language**.
- After collecting, **System analyst to document the requirements in technical language**.
- **SRS include** Data Flow Diagram, Data Dictionary, Entity Relationship diagram.

SRS defines:

- How the intended **software will interact with hardware**.
- **External interface / GUI**.
- **Speed of operation, Response time** of system.
- **Portability of software** across various platforms.
- **Maintainability**.
- **Speed of recovery** after crashing.
- **Security, Quality, Limitations** etc.

Step 3 : Software Requirement Validation

- After **SRS developed**, the requirements discussed in this document are **validated or tested**.
- Requirement validation done through **Requirement reviews taken by customers**, After developing **prototyping check with customers & Test case generations**.

Requirements can be the check against the following conditions -

- If they can **practically implement**.
- If they are **correct and as per the functionality** and specially of software.
- If there are **any ambiguities**.
- If they can **describe**.
- **Any changing or additional requirement**.

Step 4: Software Requirement Management

Requirement management is the **process of managing changing requirements** during the requirements engineering process.

Activities involved in Requirement Management:

1. **Tracking & Controlling Changes:** Handling **changing requirements from customers**.
2. **Version Control:** Keeping **track of different versions** of the system.
3. **Traceability:** Trace to software is **design, develop or test as per requirements or not**.
4. **Communication:** If changing requirements has any issues **contact with clients within time**.
5. **Monitoring & Reporting: Monitoring development process & report the status**.